Weakly Supervised Software Entity Recognition

Dumitrescul Eduard-Valentin[1], Huțanu Eusebiu-Ciprian[2], Mircea Andrei-Răzvan[3], and Păpușoi Rareș[4]

[1]University of Bucharest

January 17, 2026

Abstract

1 Introduction

The quality and quantity of data is the single most important factor when developing and using a state-of-the-art Large Language Model. One of the most popular use-cases of such models is writing code based on a prompt. During both training and inference, the LLM has access to enormous datasets containing code examples, which it uses to build upon and answer. We believe that adding the ability to filter and group this data can provide a significant increase in the quality of the responses and the generated code.

This project aims to develop a Weakly Supervised Software Entity Recognition (SER) system that is trained using automatically annotated data. The goal is to bypass the need for expensive manual labeling and make use of Transformer architectures to build a system that can identify important entities within source code and technical documentation.

1.1 Background

Named Entity Recognition is a Natural Language Processing task that aims to identify specific entities —like names, dates and places —in bodies of text. The challenge of this project is to identify a set of predefined entities in text that does not possess classic linguistic features.

Manual annotation is the standard for most machine learning projects. It has the ability to generate high-quality of data, but requires large teams of specialized people. Specifically for SER, the annotation process requires domain expertise (software engineers), which makes the task incredibly expensive. Another issue is the constant changing of the existent programming languages and the creation of new ones that created the need of a constantly evolving dataset.

Weakly Supervision is the solution we propose for this problem. It replaces the manual effort of manually annotating data with training the model on noisy, imperfect of incomplete data. However, by defining rules for annotating the data, it is possible to generate large amounts of annotated data that make up for the lower quality if the labels. This approach is reusable and easily extensible.

Transformer architectures are very effective for SER because of their ability to understant large amounts of data. Specifically, **CodeBERT**, which is pretrained in such a way that it understand both natural language and programming language.

2 Methodology

2.1 Target Entities

This SER system focuses on a set of high-value 4 core entities found in the majority of programming languages.

- **LIBRARY**: Software packages or modules (e.g. *numpy*)
- **FUNCTION_METHOD**: A function or a method name (e.g. *sum()*)
- **CLASS_NAME**: A class definition or its usage (e.g *DataLoader*)
- **FILE_PATH**: Relative or absolute file paths(e.g. */etc/config*)

2.2 Data Gathering

The principal data source is an archive of **Stack Overflow posts, votes, tags and badges**. Question bodies were extracted for both **python** and **javascript** programming languages, based on their tags. The two lanugages were chosen because of their popularity, the vast amount of data available for both of them and the significant syntactic difference between them. For consistency, only questions containing between 800 and 1000 words were selected.

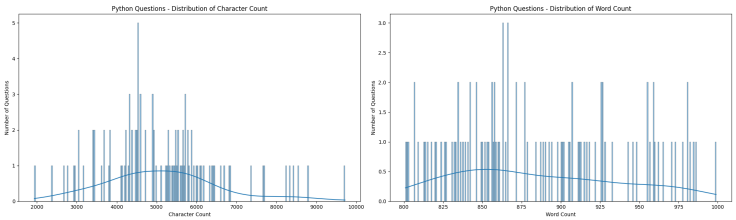


Figure 1: The distribution of character and word count based on 100 Python questions

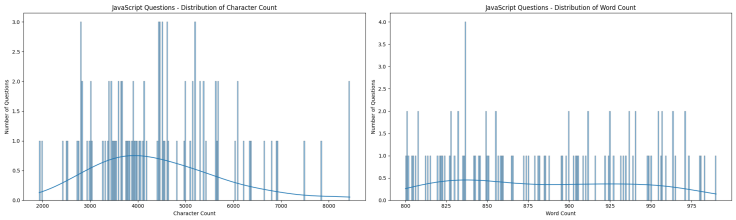


Figure 2: The distribution of character and word count based on 100 JavaScript questions

2.3 Data Preprocessing

The body of each questions contains HTML tags and code snippets. To prepare the data for SER, the code blocks were extracted and concatenated based on the HTML tags present. Only the code blocks have been selected because they contain the highest density of the defined entities. Both Pythona dn JavaScript data was preprocessed in this way.

2.4 Annotation Process

A series of **Labeling Functions** were developed for labeling the Python data for all 4 entities: LIBRARY, FUNCTION_METHOD, CLASS and FILEPATH. The LFs were built using a combination and regex and predefined keywords.

For **Library extraction**, three distinct LFs identify import patterns, covering: direct import (e.g. *import os*) aliased imports (e.g. *import pandas as pd*) partial imports (e.g. *from math import sqrt*)

For **Function and Method identification** definitions (using *def* keyword) and method and function invocation were targeted (e.g. *object.method()*).

Class detection is based on detecting class definition and instantiation patterns. To improve precision, the LFs look for PascalCase identifiers.

Filepath identification uses LFs that search for Unix-style paths (e.g. */etc/config*), Windows-style paths (e.g. *C:/Python*) and standalone filenames (e.g. *config.json*)

2.5 Tokenization

In order to prepare the data for feeding it into SER models, the *codebert-base* tokenizer was used on the preprocessed code blocks, creating the embeddings for the test. Then, the token-level labels (*IOB2 tags*) for each entity were generated.

A total of 9 tags were used: *O, B-LIBRARY, I-LIBRARY, B-CLASS_NAME, I-CLASS_NAME, B-FILE_PATH, I-FILE_PATH, B-FUNCTION_METHOD, I-FUNCTION_METHOD.*

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
Token	<s>	import	Gc	Pick	le	Gas	Gpick	le	G#	orig	inally	Gpick	le
Label	O	O	B-LIB	B-LIB	I-LIB	O	B-LIB	I-LIB	O	O	O	B-LIB	I-LIB

Table 1: Tokenization and IOB Tagging Output

3 Experimental Setup

3.1 CodeBERT

3.2 Custom Model

4 Results and Evaluation

5 Discussion

6 Conclusion